

Security Audit

Cortis (Blockchain)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	34
Conclusion	35
Our Methodology	36
Disclaimers	38
About Hashlock	39

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Cortis team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Cortis is your entire executive team, rebuilt as AI. Not just research. A full AI C-suite that thinks, debates, and delivers. A multi-agent pipeline goes head to head with itself to search, analyze, and stress-test every angle, then ships boardroom-ready output: investment memos, whitepapers, business plans, pitch decks, and financial models. The work of an analyst bench, a strategy team, and a CFO, running around the clock. Built on BNB Chain and powered by the \$COR token, Cortis runs a usage-based economy where research spending fuels staking rewards and platform growth.

Project Name: Cortis

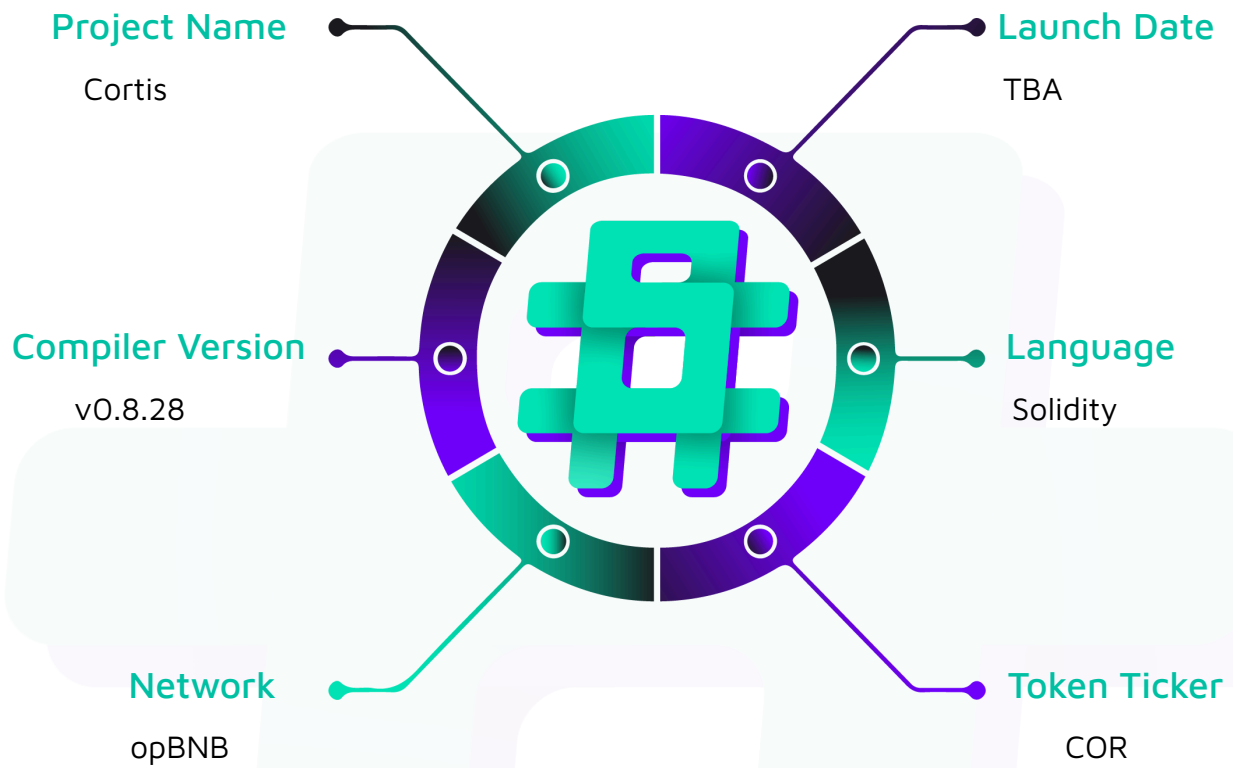
Project Type: Blockchain

Compiler Version: 0.8.28

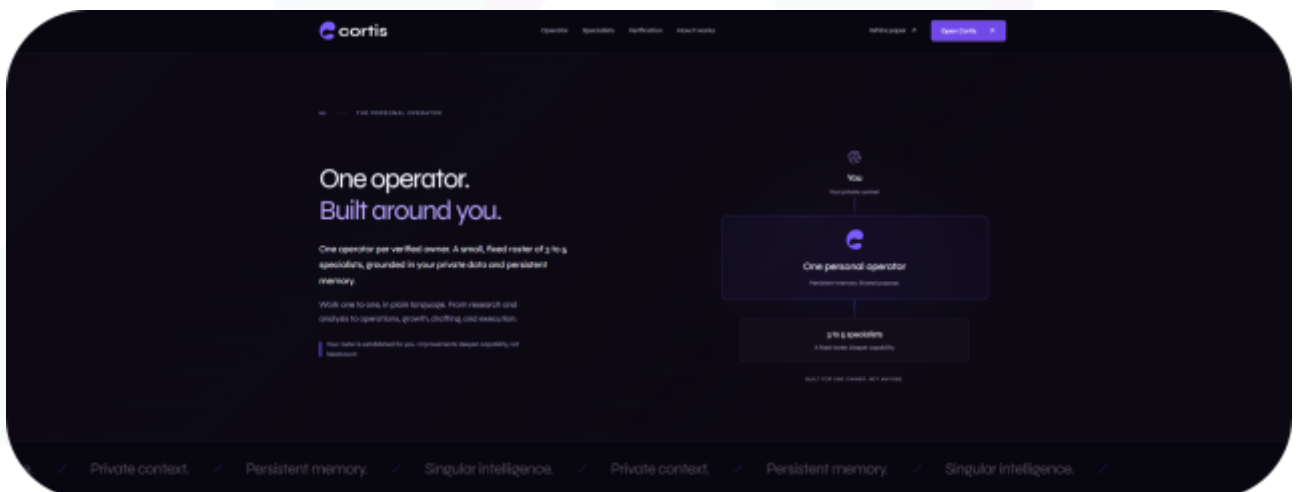
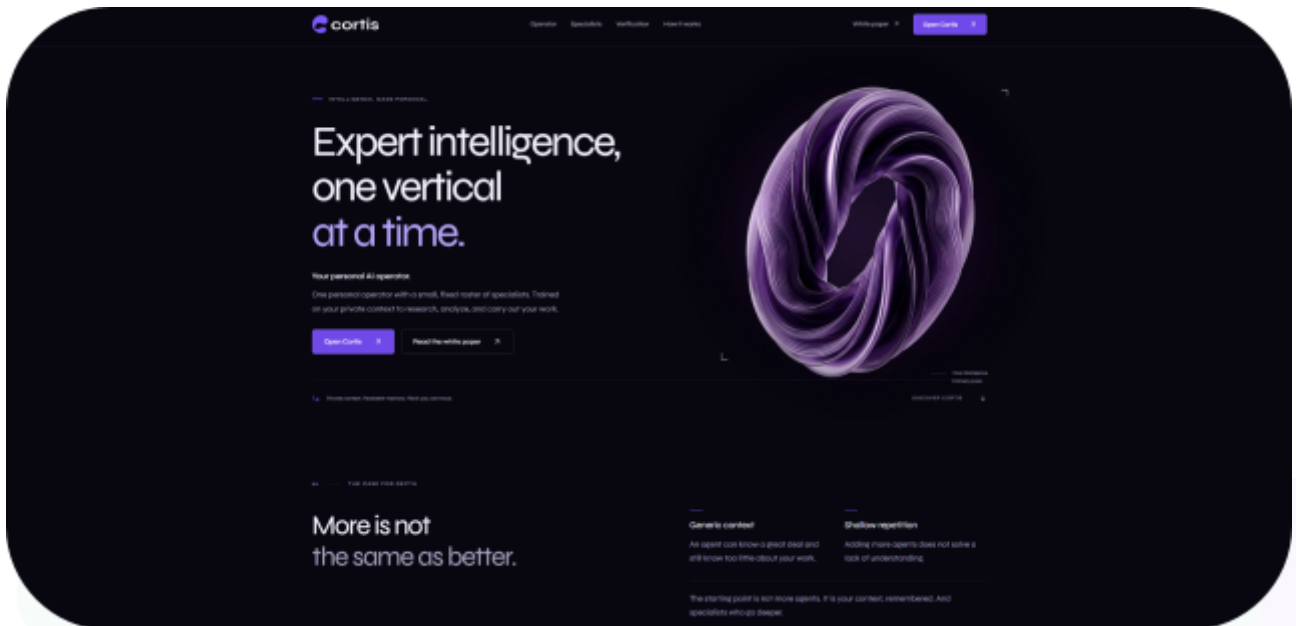
Website: <https://cortisai.com/>

Logo:



Visualised Context:

Project Visuals



Audit Scope

We at Hashlock audited the solidity code within the Cortis project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Cortis Smart Contracts
Network	opBNB
Language	Solidity
Audit Date	September, 2026
Contract 1	CortisEngagement.sol
Contract 2	AgentPassport.sol
Contract 3	AttestorRegistry.sol
Contract 4	Policies.sol
Audited GitHub Commit Hash	7d02cfefb1b6fc81c02cfa52e3972fd450f3905f
Fix Review GitHub Commit Hash	a7463692ff53f42c0ccdee5c44bfbceef3c093b2

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

1 Medium severity vulnerability

6 Low severity vulnerabilities

3 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
CortisEngagement.sol - Allows wallets and active agents to perform daily check-ins and earn points. - Records attestor-signed agent actions and supports signed outcome corrections. - Allows agents to spend points to activate time-limited boosts.	Contract achieves this functionality.
AgentPassport.sol - Mints non-transferable ERC-721 passports using issuer-signed vouchers. - Limits each owner to five simultaneously active passports. - Allows owners to deactivate passports and update their specialization.	Contract achieves this functionality.
AttestorRegistry.sol - Maintains the list of authorised protocol attestors. - Allows administrators to add attestors and guardians to revoke them. - Tracks the active, revoked, and compromised status of attestor addresses.	Contract achieves this functionality.
Policies.sol - Implements the protocol's fee-policy interface. - Provides a default policy that returns zero fees.	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Cortis project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Cortis project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] src/CortisEngagement.sol#correctActionOutcome - Debt can be abandoned by replacing the passport

Description

Correction debt is scoped to an individual `agentId`. An owner can retire the indebted passport and, after receiving a new issuer-signed mint voucher, direct all future activity to a replacement passport whose debt starts at zero.

Vulnerability Details

When a completed action is corrected after its points have already been spent, the resulting shortfall is recorded only against the affected agent:

```
mapping(uint256 agentId => uint64) public pointDebt;

if (previous == Outcome.Completed && newOutcome != Outcome.Completed) {
    a.completedActionCount -= 1;
    a.lifetimePoints -= COMPLETED_ACTION_POINTS;
    if (a.points >= COMPLETED_ACTION_POINTS) {
        a.points -= COMPLETED_ACTION_POINTS;
    } else {
        pointDebt[agentId] += COMPLETED_ACTION_POINTS - a.points;
        a.points = 0;
    }
}
```

Future awards repay only the debt stored under that same `agentId`:

```
function _creditAgent(uint256 agentId, uint64 award) private {
    AgentState storage a = _agents[agentId];
    a.lifetimePoints += award;
    uint64 repaid = pointDebt[agentId] < award ? pointDebt[agentId] : award;
```

```

    pointDebt[agentId] -= repaid;
    a.points += award - repaid;
}

```

The owner can permanently deactivate the indebted passport and free an active-roster slot:

```

function deactivate(uint256 agentId) external {
    Passport storage p = _requirePassport(agentId);
    if (p.owner != msg.sender) revert NotPassportOwner(agentId, msg.sender);
    if (!p.active) revert AlreadyInactive(agentId);

    p.active = false;
    p.deactivatedAt = uint32(block.timestamp);
    unchecked {
        _activeCount[msg.sender] -= 1;
    }
    emit PassportDeactivated(agentId, msg.sender);
}

```

A subsequently minted passport receives a new `agentId`. Because `_agents` and `pointDebt` are mappings keyed by that identifier and no migration occurs, the replacement begins with zero points, history, and debt. The replacement does require a fresh issuer-authorized voucher; the owner cannot mint it unilaterally.

In addition, a boost purchased with the subsequently invalidated points remains time-active:

```

function boostActive(uint256 agentId) external view returns (bool) {
    return _agents[agentId].boostUntil > block.timestamp;
}

```

Consider the following scenario:

1. Passport A earns and spends 100 points on a boost.
2. Corrections create 100 debt on A.
3. The owner deactivates A.

4. The issuer authorizes passport B.
5. Future awards on B do not repay A's debt, while A's boost remains active until expiry.

Impact

The correction-debt mechanism can fail to recover invalidated credit from an owner who abandons the affected identity.

Recommendation

Define the intended debt principal explicitly. If debt follows the owner, track it by owner address or require every award belonging to that owner to service aggregate debt. If debt follows an agent lineage, add an issuer-controlled replacement mechanism that migrates unresolved debt to the canonical replacement.

Status

Resolved

Low

[L-01] src/AgentPassport.sol#getApproved - `getApproved()` succeeds for nonexistent token IDs

Description

`AgentPassport.getApproved()` always returns the zero address, including when the requested passport was never minted.

Vulnerability Details

The override removes the token-existence check implemented by OpenZeppelin ERC-721:

```
function getApproved(uint256) public pure override returns (address) {  
    return address(0);  
}
```

Returning zero is appropriate for an existing soulbound passport with no approval, but ERC-721 requires `getApproved(tokenId)` to revert for an invalid NFT identifier. The current behavior makes nonexistent passports indistinguishable from existing unapproved passports through this standard getter.

Impact

ERC-721 integrations can accept a successful response as evidence that a token exists, producing incorrect indexing or validation results.

Recommendation

Restore the inherited existence validation before returning zero:

```
function getApproved(uint256 tokenId)  
    public  
    view  
    override  
    returns (address)  
{
```

```
_requireOwned(tokenId);  
return address(0);  
}
```

Status

Resolved



[L-02] `src/AttestorRegistry.sol#revokeAttestor` - A revoked attestor cannot subsequently be marked compromised

Description

The guardian can record compromise status only while revoking an active attestor. It cannot update the historical record if evidence of compromise appears after the key was already revoked.

Vulnerability Details

`revokeAttestor()` rejects every inactive key before writing the `compromised` field:

```
function revokeAttestor(address attestor, bytes32 reasonHash, bool compromised)
    external
    onlyRole(GUARDIAN_ROLE)
{
    AttestorRecord storage r = _records[attestor];
    if (!r.active) revert NotActive(attestor);

    r.active = false;
    r.compromised = compromised;
    r.revokedAt = uint32(block.timestamp);

    unchecked {
        activeAttestorCount -= 1;
    }
    emit AttestorRevoked(attestor, reasonHash, compromised);
}
```

For example, a key may first be revoked as part of routine rotation and only later be found compromised. A second call that attempts to record the new incident information reverts with `NotActive`.

Re-adding the same address also resets its current compromise marker:

```
r.active = true;
r.everActive = true;
r.compromised = false;
```

```
r.addedAt = uint32(block.timestamp);  
r.revokedAt = 0;
```

There is no separate operation for marking an inactive historical key as compromised and no durable `everCompromised` field.

Impact

Incident-response systems that rely on `recordOf(attestor).compromised` can miss a later determination that historical certificates require review.

Recommendation

Add a guardian-authorized function that can mark a previously active key as compromised independently of its current active status and emits a dedicated event.

Status

Resolved

[L-03] src/AgentPassport.sol#constructor - Constructors accept zero privileged-role addresses

Description

Required privileged-role holders are not consistently checked against the zero address before roles are granted.

Vulnerability Details

AgentPassport grants all initial roles without validating any supplied address:

```
constructor(address timelock, address issuer, address guardian)
    ERC721("C TEST Agent Passport", "CTEST-AP")
    EIP712("CortisAgentPassport", "1")
{
    _grantRole(DEFAULT_ADMIN_ROLE, timelock);
    _grantRole(ISSUER_ROLE, issuer);
    _grantRole(GUARDIAN_ROLE, guardian);
}
```

AttestorRegistry similarly omits validation for both initial role holders:

```
constructor(address timelock, address guardian) {
    _grantRole(DEFAULT_ADMIN_ROLE, timelock);
    _grantRole(GUARDIAN_ROLE, guardian);
}
```

CortisEngagement validates its guardian and dependency code, but still accepts a zero administrator:

```
if (
    guardian == address(0)
    || passport.code.length == 0
    || attestors.code.length == 0
    || (nullFeePolicy != address(0) && nullFeePolicy.code.length == 0)
) revert InvalidConfiguration();

_grantRole(DEFAULT_ADMIN_ROLE, timelock);
_grantRole(GUARDIAN_ROLE, guardian);
```

OpenZeppelin `_grantRole()` doesn't reject `address(0)`, so deployment succeeds with inaccessible roles.

Impact

A zero administrator can permanently remove role administration and unpause capability.

This requires a trusted deployment mistake rather than an unprivileged attack.

Recommendation

Reject `address(0)` for every required initial role holder in every constructor.

Status

Resolved

[L-04] src/CortisEngagement.sol#actionOf - Unknown action IDs decode to the valid default outcome Completed

Description

The getter returns a zero-initialized `ActionRecord` for an absent mapping key, while the zero value of the `Outcome` enum represents `Completed`.

Vulnerability Details

`Completed` is the first enum member and therefore has numeric value zero:

```
enum Outcome {  
    Completed,  
    Partial,  
    Failed,  
    Cancelled  
}
```

Action records and existence are tracked separately:

```
mapping(bytes32 actionId => ActionRecord) private _actions;  
mapping(bytes32 actionId => bool) private _actionExists;
```

The public getter doesn't consult `_actionExists`:

```
function actionOf(bytes32 actionId) external view returns (ActionRecord memory) {  
    return _actions[actionId];  
}
```

For an unknown `actionId`, Solidity returns an `ActionRecord` with `agentId == 0`, `outcome == Outcome.Completed`, and `recurring == false`.

Impact

An integration that reads only `actionOf(id).outcome` can interpret a nonexistent action as successfully completed. Careful consumers can notice the zero `agentId`, but the getter doesn't enforce that check and exposes a valid business-state enum value for missing data.

Recommendation

Revert when the action doesn't exist:

```
function actionOf(bytes32 actionId) external view returns (ActionRecord memory) {  
    if (!_actionExists[actionId]) revert UnknownAction(actionId);  
    return _actions[actionId];  
}
```

Alternatively, reserve enum value zero for `Unknown` or include an explicit `exists` field in the returned record.

Status

Resolved

[L-05] `src/CortisEngagement.sol#stateOf` - The documented complete public agent record is incomplete

Description

`stateOf()` is documented as returning the complete v1 public record for an agent, but it returns only the aggregate `AgentState` struct.

Vulnerability Details

The getter and its claim are:

```
/// @notice The complete v1 public record for an agent (§14.1).
function stateOf(uint256 agentId) external view returns (AgentState memory) {
    return _agents[agentId];
}
```

`AgentState` contains aggregate check-in, point, action-count, boost, and telemetry values:

```
struct AgentState {
    uint32 lastCheckInDay;
    bool everCheckedIn;
    uint16 currentStreak;
    uint16 bestStreak;
    uint32 checkInCount;
    uint64 points;
    uint64 lifetimePoints;
    uint64 actionCount;
    uint64 completedActionCount;
    uint64 actionNonce;
    uint32 boostUntil;
    bytes32 actionHead;
    uint64 mapCount;
    uint64 workflowCount;
}
```

Material state and provenance are kept elsewhere or not retained at all:

```
mapping(bytes32 actionId => ActionRecord) private _actions;
```

```
mapping(bytes32 actionId => uint64) public correctionNonces;
mapping(uint256 agentId => uint64) public pointDebt;
mapping(uint256 agentId => mapping(bytes32 runIdHash => bool)) public isRunAttested;
```

The returned struct doesn't include `pointDebt`, individual action records, correction nonces or history, run IDs, certificate signer, attester authorization epoch, specialization at action time, wallet-scoped activity, or several signed certificate fields.

Some omitted values can be obtained through separate getters or events. Others, such as signer attribution and the full signed certificate, require transaction calldata or are not represented as direct contract state. Wallet activity is deliberately owner-scoped rather than agent-scoped.

Impact

Integrators relying on `stateOf()` as the complete record can omit debt, provenance, corrections, specialization context, or detailed action history.

Recommendation

Narrow the NatSpec claim to describe `stateOf()` as the aggregate agent-state snapshot. Document the additional getters, events, and calldata needed for a complete reconstruction. If a single complete interface is required, add a versioned composite view or paginated record API and retain the missing fields.

Status

Resolved

[L-06] `src/CortisEngagement.sol#actionOf` - Action records cannot be enumerated from contract state

Description

The `ActionRecord` comment says the completed count must be recomputable from state rather than event logs, but action records are stored in a non-enumerable mapping.

Vulnerability Details

The source states:

```
/// @dev One cold slot per attested action. Unavoidable: correctActionOutcome
///      has to know what it is reversing, and the completed count has to be
///      recomputable from state rather than from event logs.
struct ActionRecord {
    uint240 agentId;
    Outcome outcome;
    bool recurring;
}
```

Records are keyed by a hash and no array or nonce-to-action index is stored:

```
mapping(bytes32 actionId => ActionRecord) private _actions;

function actionOf(bytes32 actionId) external view returns (ActionRecord memory) {
    return _actions[actionId];
}
```

The caller must already know each `actionId`. `actionCount` reveals only the number of records, and `actionHead` is a rolling hash that cannot be reversed to discover the identifiers. Consequently, the current `completedActionCount` can be read directly from `AgentState`, but it cannot be independently recomputed by enumerating `ActionRecord` values from state alone.

Impact

Independent state verification, recovery, and migration still require event or calldata history to discover action identifiers. This does not break runtime corrections because

`correctActionOutcome()` receives a known `actionId`, but it contradicts the stated state-recomputation property.

Recommendation

Either reword the comment to state that the aggregate completed count is maintained directly in storage and known individual actions can be corrected, or add an enumerable index such as:

```
mapping(uint256 agentId => mapping(uint64 nonce => bytes32 actionId))  
    public actionIdByNonce;
```

If events are intentionally the canonical discovery mechanism, document that dependency explicitly.

Status

Resolved

QA

[Q-01] src/CortisEngagement.sol#boostActive - Retired passports continue reporting active boosts

Description

boostActive() checks only the boost expiry timestamp and does not verify that the corresponding passport remains active.

Vulnerability Details

A passport owner can deactivate an active passport at any time:

```
function deactivate(uint256 agentId) external {
    Passport storage p = _requirePassport(agentId);
    if (p.owner != msg.sender) revert NotPassportOwner(agentId, msg.sender);
    if (!p.active) revert AlreadyInactive(agentId);

    p.active = false;
    p.deactivatedAt = uint32(block.timestamp);
    unchecked {
        _activeCount[msg.sender] -= 1;
    }
    emit PassportDeactivated(agentId, msg.sender);
}
```

The engagement contract does not receive a deactivation callback, and its boost getter does not consult the passport:

```
function boostActive(uint256 agentId) external view returns (bool) {
    return _agents[agentId].boostUntil > block.timestamp;
}
```

An owner can therefore activate a seven-day boost, immediately deactivate the passport, and continue receiving true from boostActive() until the stored timestamp expires. Other owner-gated engagement functions correctly reject the inactive passport through _requireActiveOwnedPassport().

Impact

No supplied on-chain path grants value solely from this getter. However, a downstream service that treats `boostActive()` as a complete entitlement decision can grant ranking, access, fee, or capability benefits to a retired identity.

Recommendation

Include passport status in the entitlement getter:

```
function boostActive(uint256 agentId) external view returns (bool) {  
    return PASSPORT.isActive(agentId)  
        && _agents[agentId].boostUntil > block.timestamp;  
}
```

Status

Resolved

[Q-02] docs/ARCHITECTURE.md#CortisEngagement-NullFeePolicy - The documented fee-policy call doesn't exist

Description

The architecture document describes an optional guarded call from the engagement core to `NullFeePolicy`, but the implementation deliberately performs no external policy call.

Vulnerability Details

The architecture diagram labels the relationship as `feePolicy.call (try/catch, no-op)`, and the accompanying text says:

> `feePolicy` points at `NullFeePolicy` pre-TGE. Any optional settlement call is wrapped so that a policy failure can never undo the base attestation or a valid boost.

In the contract, `_tryTokenEnhancement()` explicitly states that no settlement or external policy call occurs:

```
/**
 * @dev Reserved token path. This revision performs no settlement and makes
 *      no optional external policy call.
 */
function _tryTokenEnhancement(uint256 agentId, bytes32 actionId, uint256) private {
    // Fee/lock settlement is absent from this pre-TGE source. Avoid calling
    // an optional external policy which can exhaust gas or return huge data.
    emit TokenEnhancementSkipped(
        agentId,
        actionId,
        keccak256("token settlement not implemented")
    );
}
```

Although the contract stores `feePolicy`, neither the attestation path nor the boost path invokes `IFeePolicy.attestationFee()` or any other policy method.

Impact

The documented integration architecture doesn't match deployed behavior.

Recommendation

Choose and document one design.

Status

Resolved

[Q-03] src/CortisEngagement.sol#setFeePolicy - Policy setters cannot activate post-TGE functionality

Description

The deployment checklist says that future fee and tier policies can be attached without replacing the immutable core, but the setter functions only store addresses that the relevant execution paths never invoke.

Vulnerability Details

The setters validate code presence and update storage:

```
function setFeePolicy(address newPolicy) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (newPolicy != address(0) && newPolicy.code.length == 0) {
        revert InvalidConfiguration();
    }
    emit FeePolicyUpdated(feePolicy, newPolicy);
    feePolicy = newPolicy;
}

function setCorTierPolicy(address newPolicy) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (newPolicy != address(0) && newPolicy.code.length == 0) {
        revert InvalidConfiguration();
    }
    emit CorTierPolicyUpdated(corTierPolicy, newPolicy);
    corTierPolicy = newPolicy;
}
```

The attestation enhancement always emits a skip event:

```
function _tryTokenEnhancement(uint256 agentId, bytes32 actionId, uint256) private {
    emit TokenEnhancementSkipped(
        agentId,
        actionId,
        keccak256("token settlement not implemented")
    );
}
```

The boost path also skips settlement even when `corTierPolicy` is configured:

```

uint256 corLocked = 0;
if (useCorEnhancement) {
    if (corTierPolicy == address(0)) {
        emit TokenEnhancementSkipped(
            agentId,
            bytes32(0),
            keccak256("token layer not active")
        );
    } else {
        emit TokenEnhancementSkipped(
            agentId,
            bytes32(0),
            keccak256("token settlement not implemented")
        );
    }
}
}

```

No setter can inject executable logic into an immutable contract unless the core already calls the configured module.

Impact

Deploying a real policy and calling the setter cannot introduce attestation fees, token locks, tiers, or token-enhanced boosts. The documented no-redeployment post-TGE activation path is therefore nonfunctional and will require a new engagement-core deployment or implementation changes before the current source is deployed.

Recommendation

Before production deployment, either implement and audit explicit policy interfaces and calls for each promised feature or remove the claim that setters can activate post-TGE functionality without redeployment. Also, consider defining the cross-chain model for a BSC \$COR token consumed by opBNB contracts.

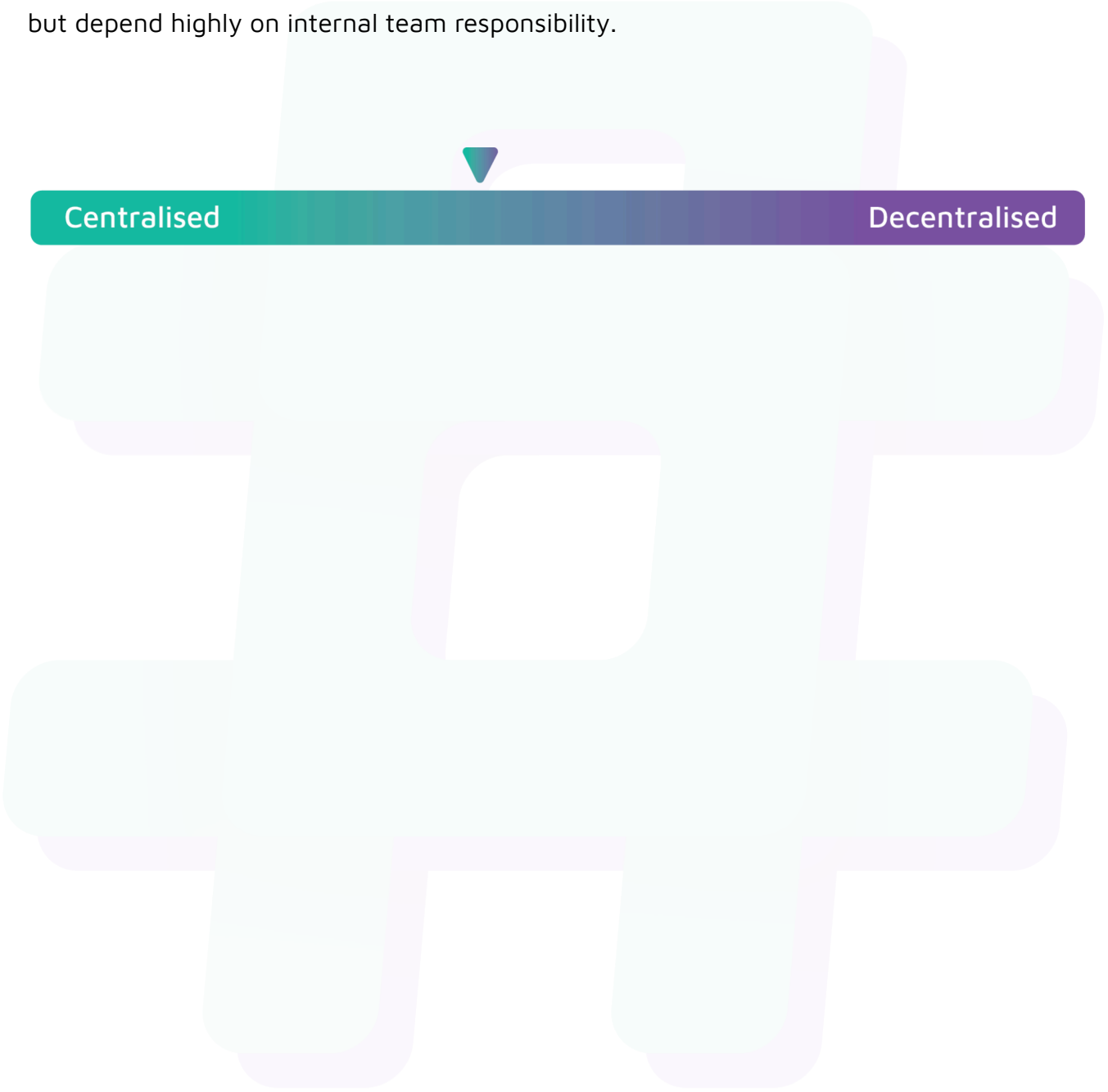
Status

Resolved

Centralisation

The Cortis project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Cortis project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.